

IoT Devices – Volledige Examensamenvatting

Deze samenvatting bevat **de volledige leerstof** die nodig is om te slagen voor:

- **Deel 1: theorie-examen** (open vragen, gesloten boek)
- **Deel 2: praktisch programmeerexamen** (HAL, STM32CubeMX, Keil)

De uitleg is bewust **uitgebreid en verhalend**, omdat bij open vragen verwacht wordt dat je **principes helder kan uitleggen**, niet enkel definities kan opsommen.

Hoofdstuk 1 – Inleiding, Microcontrollers en HAL

Doel van de cursus

Deze cursus bouwt verder op het vak *Microcontrollers* en focust op:

- STM32L476RG microcontroller
- Programmeren in **C**
- Gebruik van de **HAL-bibliotheken**
- Efficiënte IO via **polling, interrupts en DMA**
- Communicatie via **UART, I²C, SPI en WiFi**
- Introductie tot **RTOS**

Een **microcontroller** is een volledig klein computersysteem op één chip:

- CPU (Cortex-M4 in de STM32L476RG)
- Flashgeheugen (programma)
- RAM (data)
- Peripherals: timers, ADC, UART, I²C, SPI, GPIO, ...

In embedded systemen is er meestal **geen besturingssysteem zoals Windows**, maar draait jouw C-programma rechtstreeks op de hardware. Daardoor moet jij zelf:

- IO configureren (pinnen, snelheden, pull-ups, ...)
 - Peripherals initialiseren (UART, timers, ADC, ...)
 - De flow van het programma bepalen (superloop, interrupts, RTOS)
-

CMSIS versus HAL

CMSIS (Cortex Microcontroller Software Interface Standard)

Bij CMSIS programmeer je rechtstreeks op **register-niveau**.

Dit levert **zeer efficiënte en snelle code**, maar maakt de software sterk afhankelijk van het exacte microcontrollertype.

HAL (Hardware Abstraction Layer)

HAL vormt een **abstractielaag boven CMSIS** en verbergt hardware-details. Dit maakt de code:

- Leesbaarder
- Onderhoudsvriendelijker
- Eenvoudiger te migreren naar andere STM32-microcontrollers

Nadeel: iets minder efficiënt dan registerprogrammering.

Examenvraag

Leg het verschil uit tussen CMSIS en HAL en geef een situatie waarin HAL een betere keuze is.

Praktisch voorbeeld

Stel dat je een GPIO-pin als output wil instellen:

- Met CMSIS moet je zelf in de referentiemanual opzoeken **welk register** en **welke bits** je moet aanpassen (bv. `MODER`, `OTYPER`, `PUPDR` ...) en met bitmaskers werken.
- Met HAL gebruik je een **struct** (`GPIO_InitTypeDef`) en een functie zoals `HAL_GPIO_Init()`; CubeMX genereert die code automatisch.

HAL is dus ideaal voor **snelle ontwikkeling, leesbaarheid en foutreductie**. CMSIS is bruikbaar als je **maximale prestatie** en **volledige controle** nodig hebt (bv. in een tijdkritische ISR of bij zeer beperkte geheugenruimte).

Afbeelding: lagenmodel CMSIS–HAL–User Code

De HAL-bibliotheek is een softwarelaag (C-taal) die bovenop de CMSIS-bibliotheken ligt, waardoor je onafhankelijk(er) wordt van het type microcontroller.



STM32-projectstructuur (CubeMX → Keil)

In deze cursus werk je typisch in **drie stappen**:

1. Hardwareconfiguratie in STM32CubeMX

- Klokconfiguratie (HSE/HSI, PLL, systeem snelheid)
- GPIO-modi, pull-ups/pull-downs
- Peripherals (UART, I²C, SPI, timers, ADC, DMA, ...)
- NVIC-prioriteiten

2. Codegeneratie

CubeMX genereert een volledig **HAL-project**:

- `Core/Src/main.c` – je eigen applicatiecode
- `Core/Src/stm32l4xx_it.c` – interrupt handlers
- `Core/Src/stm32l4xx_hal_msp.c` – low-level init
- `Drivers/STM32L4xx_HAL_Driver` – HAL-broncode

3. Uitwerken van de applicatielogica in Keil µVision

- Je vult de lege callbacks (`HAL_xxx_MspInit`, `HAL_xxx_TxCpltCallback`, ...) en de `while (1)` in
- Je gebruikt de gegenereerde `MX_..._Init()` functies om peripherals te initialiseren

Zorg dat je het **verschil** begrijpt tussen gegenereerde code (niet zomaar wijzigen) en **user code blokken** waarin jij de logica invult.

Hoofdstuk 2 – C-taal (Embedded focus)

C-strings

In C bestaat een string uit een **array van karakters** die eindigt op een **nulkarakter (\0)**.

Gebruik:

- UART-communicatie
- `printf()`-uitvoer

Een string zoals "TEST" wordt in het geheugen opgeslagen als:

- `{ 'T', 'E', 'S', 'T', '\0' }` → totale lengte = 5 karakters

Het **nulkarakter** geeft aan waar de string eindigt. Functies zoals `strlen()` of `printf()` lopen de array af **tot ze \0 tegenkomen**. Staat er geen `\0`, dan leest de functie verder in het geheugen → **onvoorspelbaar gedrag (undefined behavior)**.

Veelgebruikte functies (uit `string.h`):

- `strlen(s)` – lengte van de string (zonder `\0`)
- `strcpy(dst, src)` – string kopiëren
- `strcat(dst, src)` – strings aan elkaar plakken
- `strcmp(a, b)` – strings vergelijken

In embedded context moet je **voorzichtig zijn met geheugen**:

- Vermijd te grote buffers op de stack
- Let op dat je altijd voldoende ruimte voorziet voor **alle karakters + \0**

Examenvraag

Waarom is het nulkarakter (`\0`) noodzakelijk bij C-strings?

Afbeelding: string in geheugen

Om aan te geven dat de tekst stopt, wordt er op het einde een 'nulkarakter' toegevoegd (`\0`).

V I V E S \0

PDF p. ±37–38

`const`, `volatile` en embedded C

In embedded C komen de kwalificaties `const` en `volatile` **zeer vaak** voor:

- `const` betekent dat je de variabele **niet mag wijzigen** in je code.
Wordt gebruikt voor:
 - Look-up tabellen in Flash
 - Configuratieconstanten (bv. `const uint32_t BAUDRATE = 115200;`)

- `volatile` vertelt de compiler dat de waarde **asynchroon kan veranderen** (door interrupts, DMA, hardware-registers).
Zonder `volatile` kan de compiler optimalisaties doen die in embedded context **fout** zijn (bv. een variabele maar één keer lezen terwijl de hardware die ondertussen wijzigt).

Typische voorbeelden:

- Flags gezet in een ISR: `volatile uint8_t button_pressed;`
- Hardware-registerdefinities in CMSIS zijn altijd `volatile`.

Examenvraag

Leg uit waarom een variabele die in een interrupt wordt aangepast als `volatile` gedeclareerd moet worden.

Pointers en structs

- Pointer = adres van een variabele
- HAL gebruikt pointers naar structs
- Void pointers laten generiek gebruik toe

Belangrijk

Pointers moet je **begrijpen**, niet memoriseren.

Een pointer **bewaart het geheugenadres** van een variabele:

- `int x = 5;`
- `int *px = &x;` → `px` bevat het adres van `x`
- `*px` geeft de **waarde** waarnaar de pointer wijst (hier 5)

In embedded C gebruik je pointers om:

- Functies een variabele **by reference** te laten aanpassen
- Grote structs **niet te kopiëren**, maar enkel het adres door te geven
- Hardware-registers te benaderen via **pointers naar vaste adressen**

Voorbeeld uit HAL:

- `UART_HandleTypeDef huart2;`
- Functies zoals `HAL_UART_Transmit(&huart2, ...)` verwachten een **pointer naar de struct**.

Een `void *` pointer is een **generieke pointer** zonder type; de HAL gebruikt dit om bv. in callbacks één generieke functie te kunnen schrijven die met verschillende types peripherals kan werken. Voor gebruik moet je deze casten naar het juiste type.

Afbeelding: pointers naar structs

PDF p. ±41–42

Structs in HAL-configuratie

Vrijwel alle HAL-initialisatiefuncties gebruiken **configuratiestructs**:

- `GPIO_InitTypeDef` voor GPIO
- `UART_HandleTypeDef` voor UART
- `TIM_HandleTypeDef` voor timers/PWM
- `I2C_HandleTypeDef` voor I²C

Je vult de velden van zo'n struct in (mode, snelheid, pull, baudrate, ...) en geeft de **pointer** door aan een HAL-functie. Hierdoor kan HAL dezelfde API houden, zelfs als de onderliggende registers in andere STM32-families verschillen.

Belangrijk voor het examen:

- Kunnen uitleggen **waarom** een pointer wordt doorgegeven (efficiënt, geen kopieën).
- Begrijpen dat één handle (`huart2`, `htim3`, ...) alle **statusinformatie** over die peripheral bevat (state, errors, configuratie).

Hoofdstuk 3 – GPIO en Basis IO

GPIO-concepten

- Input vs output
- Pull-up en pull-down
- Actief hoog / actief laag

```
HAL_GPIO_ReadPin();  
HAL_GPIO_WritePin();
```

Examenvraag

Wat is het verschil tussen een pull-up en een pull-down weerstand?

Pull-up / pull-down

Een ingangspin mag **nooit zweven** (onbepaald niveau), want dan kan ruis of lekstroom een willekeurige waarde veroorzaken. Een pull-weerstand zorgt voor een **standaardniveau** wanneer de knop/schakelaar **niet actief** is:

- Pull-up: pin wordt standaard naar **logisch 1 (VCC)** getrokken
- Pull-down: pin wordt standaard naar **logisch 0 (GND)** getrokken

Bij een knop aan GND gebruik je typisch een **pull-up**: ongedrukt = 1, gedrukt = 0. Dit is elektrisch robuust en wordt vaak gecombineerd met een **actief lage logica** (0 = actief).

Actief hoog / actief laag

Bij **actief hoog** wordt een functie geactiveerd door een **hoog** signaal (1).

Bij **actief laag** wordt een functie geactiveerd door een **laag** signaal (0), vaak aangeduid met een streepje of **N** in de naam (bv. `RESET_N`).

Afbeelding: GPIO-instellingen in CubeMX

PDF p. ±19–20

Debouncing van knoppen

Mechanische knoppen **stuiteren**: één fysieke druk kan meerdere snelle overgangen veroorzaken (0-1-0-1-0). Zonder debouncing denkt de microcontroller dat er **meerdere drukken** zijn.

Manieren om te debouncen:

- **Hardwarematig:** RC-filter, Schmitt-trigger
- **Softwarematig:**
 - In een timer-interrupt of superloop de knop enkel **om de paar milliseconden** uitlezen
 - Een toestand pas als "geldig" beschouwen als hij meerdere opeenvolgende metingen hetzelfde is

Examenvraag

Wat is "debouncing" en waarom is het nodig bij mechanische knoppen?

Hoofdstuk 4 – IO-transfers

Polling

De CPU controleert continu of data beschikbaar is.

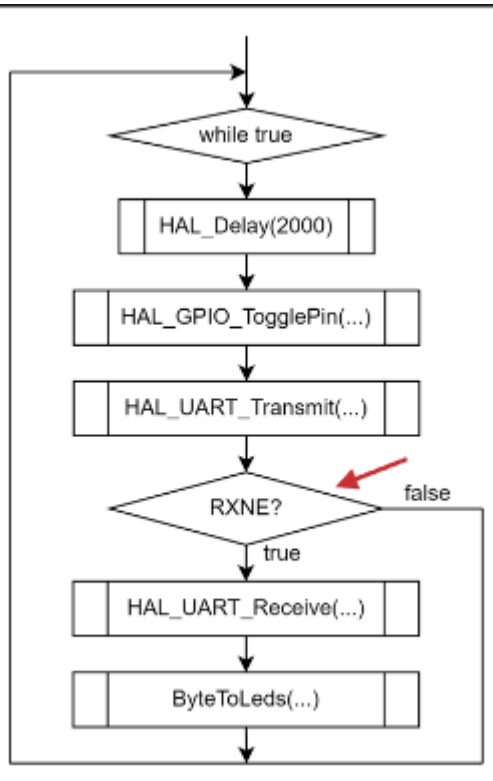
- Voordeel: eenvoudig
- Nadeel: CPU-geblokkeerd

Bij polling heb je meestal een **oneindige superloop**:

```
while (1) {
    if (data_available()) {
        handle_data();
    }
}
```

Zolang er geen data is, doet de CPU **niets nuttigs**. Dit is acceptabel voor **zeer eenvoudige applicaties**, maar in complexe systemen wil je dat de CPU **andere taken** kan uitvoeren terwijl ze wacht op IO.

Afbeelding: polling schema



PDF p. ±4-5

Interrupts

De peripheral verwittigt de CPU wanneer nodig.

- NVIC configuratie
- Interrupt handlers

Examenvraag

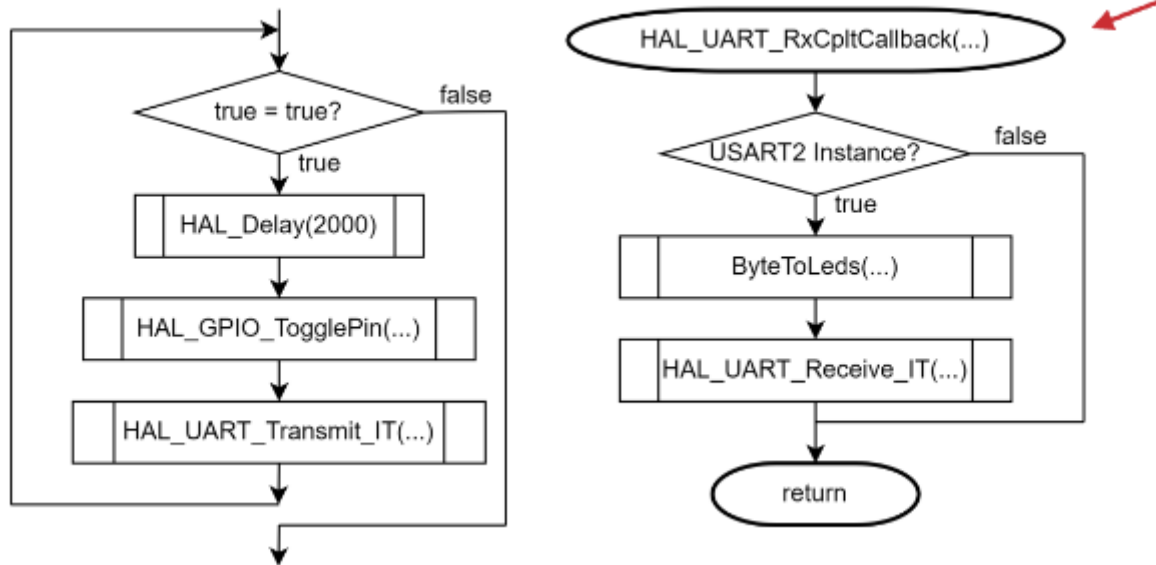
Waarom zijn interrupts efficiënter dan polling?

Bij een interrupt kan de CPU **andere code uitvoeren of in sleep gaan** totdat een gebeurtenis optreedt (bv. timer overflow, ontvangen UART-byte, druk op een knop). Wanneer de gebeurtenis optreedt:

1. De CPU onderbreekt de huidige taak
2. Voert de **ISR (Interrupt Service Routine)** uit
3. Keert terug naar de vorige taak

Belangrijke begrippen:

- **NVIC** (Nested Vectored Interrupt Controller) beheert prioriteiten van interrupts
- Hogere prioriteit → kan een lagere prioriteit onderbreken
- In een ISR moet je **zo weinig mogelijk werk** doen (snel klaar zijn) en langere verwerking doorschuiven naar de main-loop of een RTOS-taak.

Afbeelding: interrupt flow**PDF p. ±7–10**

Polling vs. Interrupt vs. DMA – overzicht

Method	CPU-belasting	Latentie	Complexiteit	Typisch gebruik
Polling	Hoog (continue busy-wait)	Afhankelijk van loop	Laag	Eenvoudige, trage events
Interrupt	Lager, enkel bij events	Laag	Gemiddeld	Knoppen, timers, UART-ontvangst
DMA	Zeer laag per byte/sample	Laag per blok	Hoger	Grote/continue datastromen

DMA (Direct Memory Access)

Data wordt rechtstreeks naar het geheugen geschreven.

- CPU enkel interrupt bij voltooiing
- Zeer efficiënt

Examenvraag

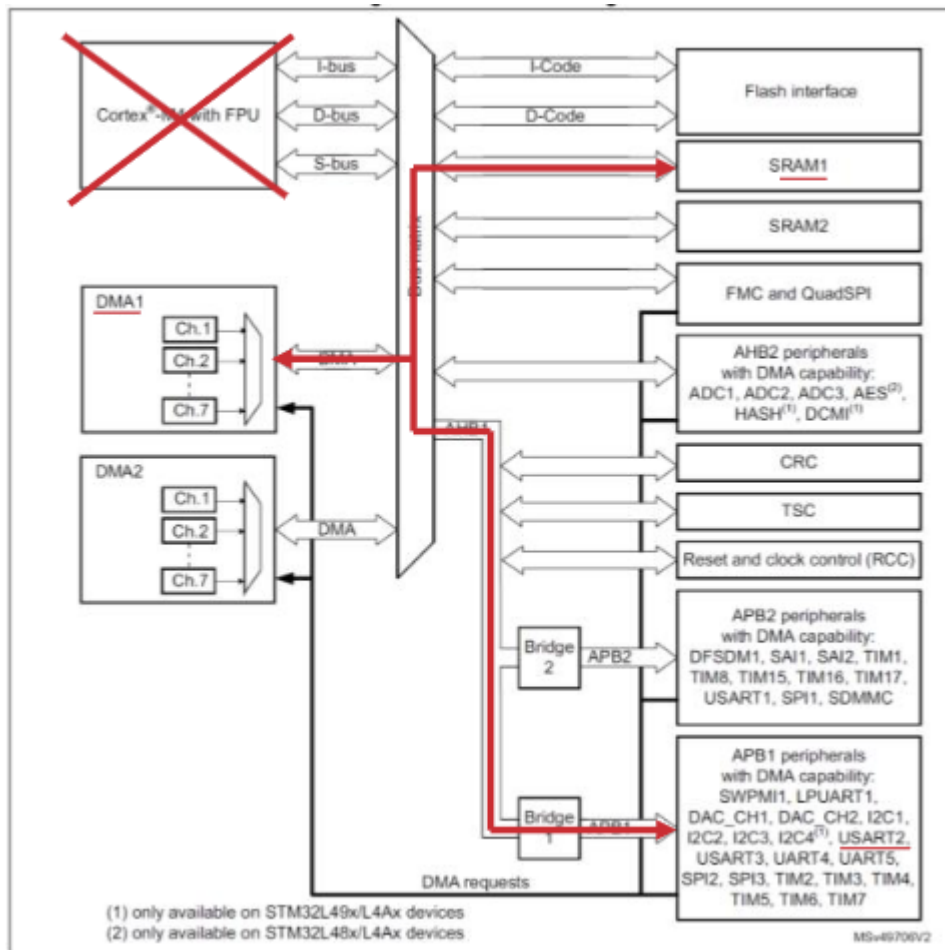
Vergelijk DMA met polling en interrupts.

DMA is ideaal voor **grote of continue datastromen** (bv. ADC-samples, seriële data, audio).

Zonder DMA moet de CPU **elke byte** zelf uit een register lezen/schrijven. Met DMA configureer je eens:

- Bronadres (bv. peripheral register)
- Doeladres (bv. buffer in RAM)
- Aantal transfers
- Eventueel **circular mode** (continu in een ringbuffer schrijven)

Daarna kan de CPU **andere zaken doen**, terwijl de DMA-controller de data verplaatst. Enkel bij **half-vol** of **vol buffer** krijg je een interrupt.

Afbeelding: DMA + bus matrix

PDF p. ±13–15

Hoofdstuk 5 – Timers en PWM

Timers

- Prescaler
- Counter
- Input Capture

Een timer telt op (of af) met een bepaalde **tijdsresolutie**. De ingangsklok (bv. 80 MHz) wordt eerst gedeeld door de **prescaler**, daarna telt de **counter** door tot een bepaalde **auto-reload waarde (ARR)**.

Formule (in eenvoudige vorm):

- $\text{timer_freq} = f_{\text{clk}} / (\text{prescaler} + 1)$
- $\text{update_freq} = \text{timer_freq} / (\text{ARR} + 1)$

Door prescaler en ARR te kiezen, stel je de **frequentie** in waarop de timer overloopt (en dus een interrupt of event kan genereren). Met **Input Capture** kun je tijdstippen van inkomende signalen meten (bv. pulsbreedte of periode van een signaal).

Afbeelding: timer clock schema

Prescaler ingesteld op 79, dit om klokfrequentie van timer 2 te laten tellen per 1 μ s.
 Uitwerking: zie p. 1184 in STM32L476RG Reference Manual:

$$\text{Counter clock frequency (CK}_{CNT}\text{)} = \frac{f_{CK_PSC}}{(PSC + 1)} = \frac{80\text{MHz}}{(79 + 1)} = 1\text{MHz} = 1\mu\text{s}$$

PDF p. ±36

Input Capture vs. Output Compare

- **Input Capture:** de timer "vangt" het **telregister** op op het moment dat een ingangspin een flank ziet. Zo kun je perioden en pulsbreedtes **met hoge resolutie** meten.
- **Output Compare:** de timer vergelijkt voortdurend de tellerstand met een **vergelijkingswaarde**. Bij een match kan hij:
 - Een uitgangspin togglen (toggle-mode)
 - Een interrupt genereren
 - Een PWM-kanaal updaten

Examenvraag

Leg het verschil uit tussen Input Capture en Output Compare bij timers en geef van elk een voorbeeldtoepassing.

PWM

Gebruikt voor servo's en motoren.

- Duty cycle
- Frequentie

Examenvraag

Hoe stuur je een servo aan met PWM?

Bij PWM blijft de **frequentie** meestal vast, en verander je de **duty cycle** (percentage "hoog" binnen één periode). De gemiddelde spanning over de belasting is ongeveer:

- $V_{\text{gemiddeld}} \approx \text{duty_cycle} * V_{CC}$

Voor een standaard RC-servo:

- Frequentie ≈ 50 Hz (periode 20 ms)
- Pulsbreedte ≈ 1 ms $\rightarrow$ uiterste hoek links
- Pulsbreedte ≈ 2 ms $\rightarrow$ uiterste hoek rechts

Je gebruikt dus een timer-kanaal in PWM-mode en stelt de **compare-waarde** zó in dat de gewenste pulsbreedte ontstaat.

Afbeelding: PWM-sigitaal
 PWM-sigitaal
PDF p. ±37Hoofdstuk 6 – I²C

I²C-bus

- SDA & SCL
- Open drain
- Pull-up weerstanden

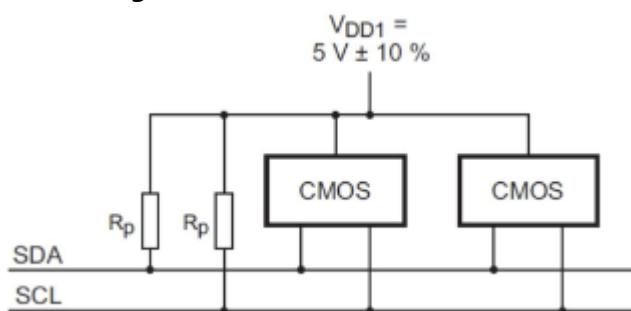
I²C is een **seriële, synchrone bus** met één of meerdere masters en meerdere slaves.

Omdat de lijnen **open-drain** zijn, kunnen meerdere toestellen de bus delen zonder dat ze elkaar kortsluiten:

- Iedereen kan enkel naar **laag** trekken
- Pull-up weerstanden trekken de lijn naar **hoog** als niemand actief is

Elke slave heeft een **uniek adres** (7- of 10-bit). De master spreekt een specifieke slave aan door dat adres in het adresframe te plaatsen.

Afbeelding: I²C bus



PDF p. ±6–7

I²C-communicatie

- Start
- Address + R/W
- ACK/NACK
- Stop

Examenvraag

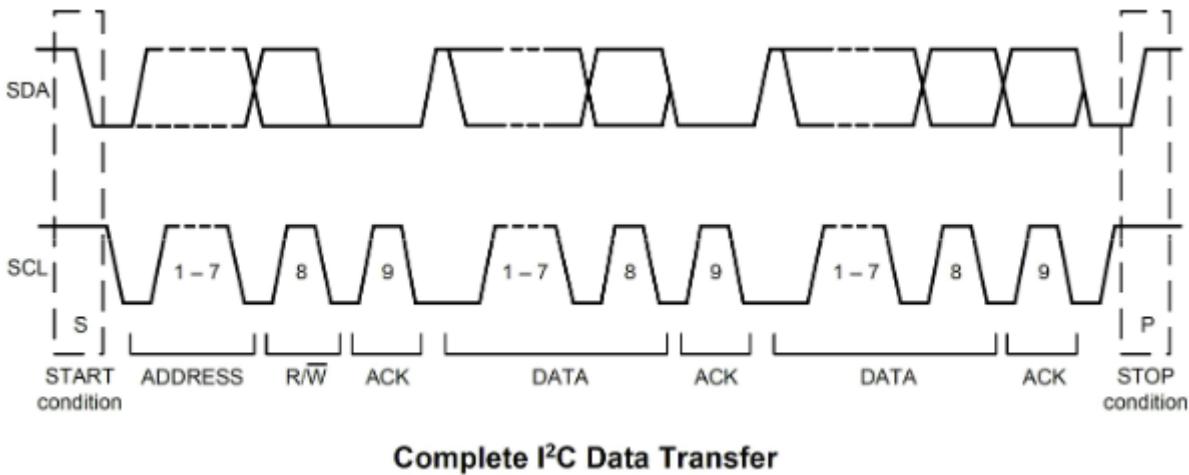
Waarom moet je bij I²C-lezen eerst schrijven?

Typisch I²C-scenario met een sensor of geheugenchip:

1. Master stuurt **Start**
2. Master stuurt **slave-adres + Write-bit**
3. Master stuurt het **registeradres** binnen de slave (bv. "temperatuurregister")
4. Master stuurt **herstart (Repeated Start)**
5. Master stuurt **slave-adres + Read-bit**
6. Slave stuurt de gevraagde data

Je "schrijft" dus eerst **welk register** je wil lezen, daarna doe je een leesoperatie. Zonder die eerste schrijffase weet de slave niet **welke interne locatie** je bedoelt.

Afbeelding: I²C timingdiagram

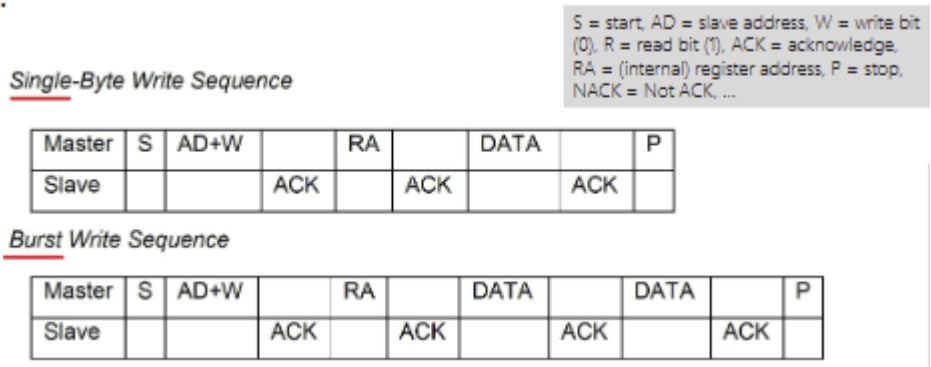


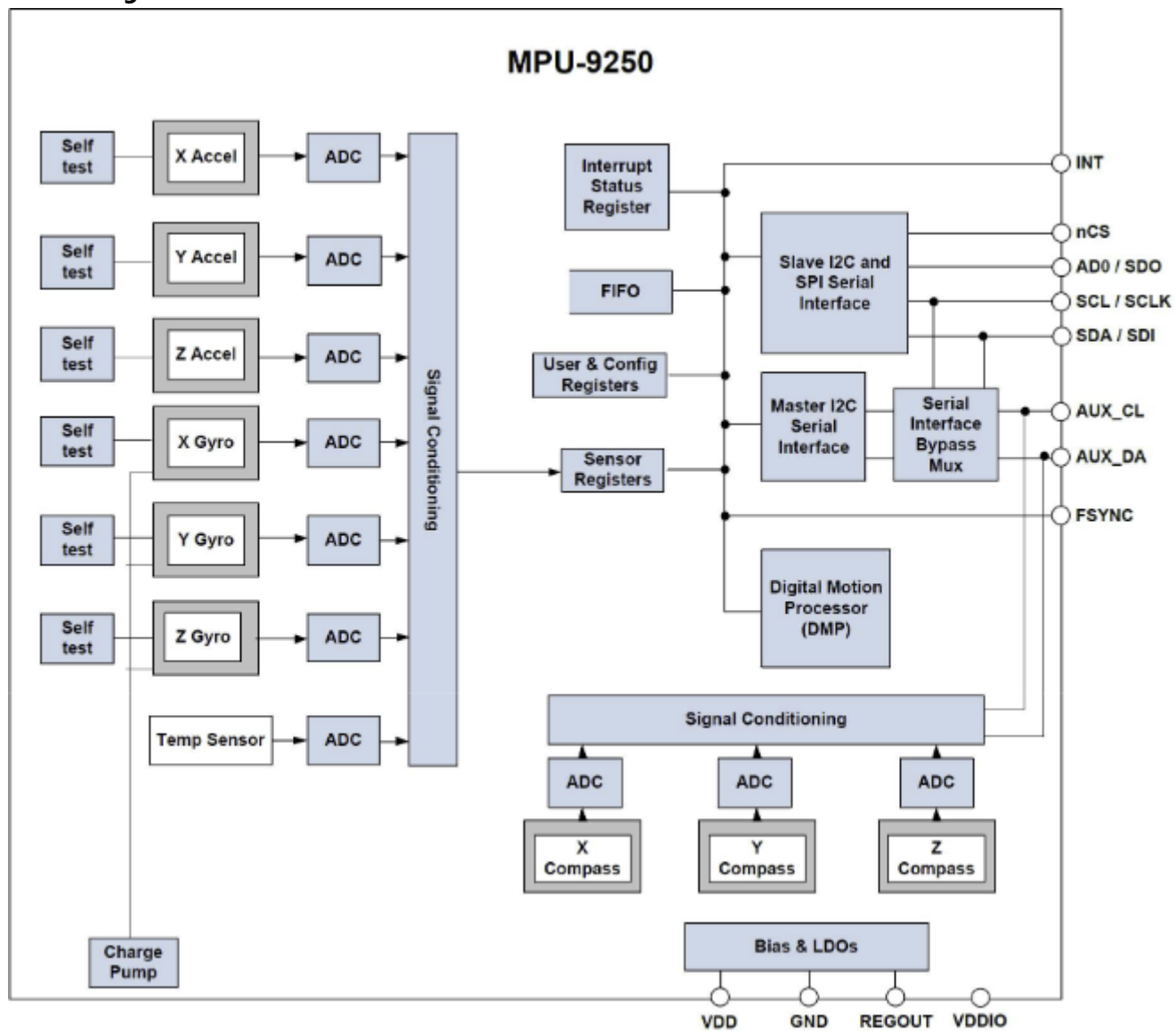
PDF p. ±9–12

Veelvoorkomende fouten bij I²C:

- Verkeerde **adresnotatie** (7-bit vs. 8-bit adres met R/W-bit)
- Vergeten van de **pull-up weerstanden** → bus blijft laag
- Slave verwacht een **Repeated Start**, maar je stuurt een gewone Stop + Start
- Verkeerde snelheidsmodus (Standard Mode, Fast Mode, ...)

Afbeelding: I²C master schrijft naar slave-register



Afbeelding: MPU9250 sensor als I²C-slave

Hoofdstuk 7 – SPI

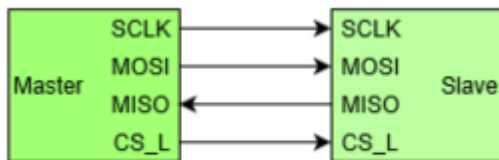
SPI-basis

- MOSI / MISO / SCK / CS
- Full-duplex

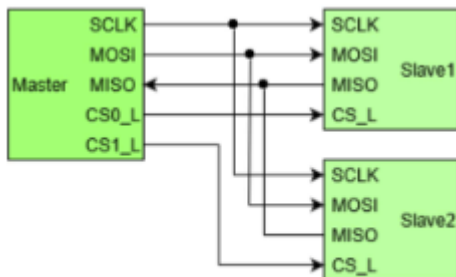
SPI is een **snelle, synchrone seriële bus** met een duidelijke master-slave structuur:

- Master genereert de klok (**SCK**)
- Data gaat van master naar slave via **MOSI** (Master Out Slave In)
- Data gaat van slave naar master via **MISO** (Master In Slave Out)
- Elke slave heeft een aparte **chip select (CS / NSS)**

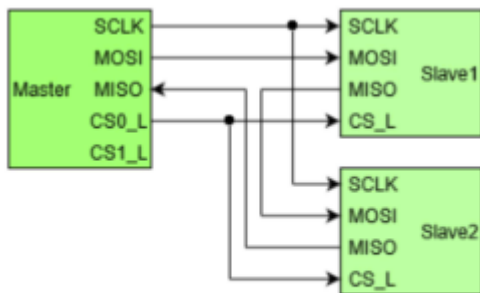
In tegenstelling tot I²C heeft SPI **geen adressen** op de bus; de master kiest een slave door zijn CS-lijn laag te maken.

Afbeelding: SPI point-to-point verbinding

SPI Bus – Simple Point-to-Point
Topology

Afbeelding: SPI ster-topologie met meerdere slaves

SPI Bus – Star Topology

Afbeelding: SPI daisy-chain verbinding

SPI Bus – Daisy Chain Topology

Hoofdstuk 8 – UART

- TX / RX
- Baudrate
- Asynchroon

UART is een **asynchrone seriële communicatie** zonder gedeelde kloklijn. In plaats daarvan spreken zender en ontvanger **baudrate, aantal databits, pariteit en stopbits** vooraf af (bv. 115200 8N1).

Een typisch UART-frame bevat:

- 1 startbit (laag)
- 5–9 databits (LSB eerst)
- Optioneel 1 pariteitsbit
- 1 of 2 stopbits (hoog)

Omdat er geen kloklijn is, gebruikt de ontvanger zijn **eigen klok** en samplet hij de lijn op vaste tijdsintervallen. Als de baudrates te veel afwijken, gaat de sampling uit fase en ontstaan bitfouten.

UART wordt vaak gebruikt voor:

- Debug-output (`printf` via een seriële terminal)
- Communicatie met modules (bv. GPS, Bluetooth, WiFi-modems)

Afbeelding: UART frame

 UART-frameopbouw

PDF p. ±11

Typische UART-problemen

- Verkeerde instellingen (baudrate, databits, pariteit) tussen zender en ontvanger
- Vergeten van **CR/LF** of verkeerde line endings in een terminal
- Bufferoverrun omdat de applicatie ontvangen data niet snel genoeg verwerkt
- Blokkeren op `HAL_UART_Receive()` in **blocking mode** in plaats van interrupt/DMA-mode te gebruiken

Hoofdstuk 9 – WiFi (ESP32-C3)

- UART communicatie
- AT-commando's
- ESP32 verzorgt netwerkstack

De STM32 fungeert hier als **host-microcontroller** en de ESP32-C3 als **WiFi-modem**.

Via UART stuurt de STM32 **AT-commando's** (tekstcommando's) naar de ESP32, zoals:

- Verbinden met een access point
- Openen van een TCP/UDP-verbinding
- Versturen/ontvangen van data

De volledige **TCP/IP-stack en WiFi-authenticatie** draait op de ESP32. Zo moet je op de STM32 geen complex netwerkprotocol implementeren, enkel de juiste commando's sturen en antwoorden verwerken.

Voorbeeld AT-commando-sequentie

Typische stappen om een TCP-verbinding op te zetten:

1. Module testen: `AT` → antwoord `OK`
2. Verbinden met WiFi: `AT+CWJAP="SSID", "PASSWORD"`
3. TCP-verbinding maken: `AT+CIPSTART="TCP", "server.com", 80`
4. Data versturen:
 - `AT+CIPSEND=length`
 - (wachten op `>` prompt)
 - HTTP-request of andere payload sturen
5. Verbinding sluiten: `AT+CIPCLOSE`

Examenvraag

Leg uit waarom de STM32 bij gebruik van de ESP32-C3 geen eigen TCP/IP-stack nodig heeft.

Hoofdstuk 10 – RTOS

Wat is een RTOS?

- Taken
- Scheduling
- Deterministisch gedrag

Examenvraag

Waarom gebruik je een RTOS?

Zonder RTOS schrijf je meestal een **superloop** met veel **if**-structuren en flags om verschillende functies om de beurt uit te voeren. Dit wordt snel onoverzichtelijk wanneer:

- Je veel verschillende periodieke taken hebt (sensor lezen, communicatie, logging, ...)
- Sommige taken **snelle responstijd** vereisen

Een **RTOS (Real-Time Operating System)** laat je toe om:

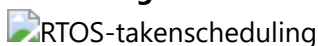
- Meerdere **taken (threads)** te definiëren
- Elke taak een **prioriteit** en eventueel een **periode** te geven
- Synchronisatie te doen via **queues, semaforen, mutexen**

Het RTOS-scheduler beslist **welke taak wanneer draait**, rekening houdend met prioriteiten. Zo kun je garanderen dat kritieke taken **binnen een voorspelbare tijd** worden uitgevoerd (deterministisch gedrag).

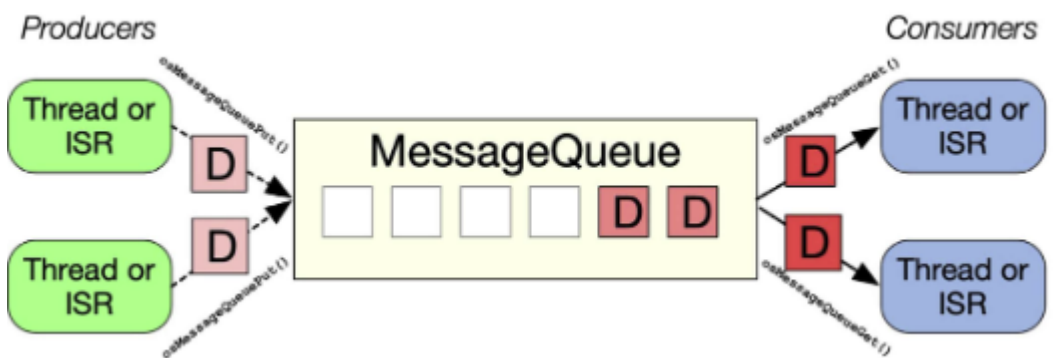
Je gebruikt een RTOS wanneer de applicatie:

- Complex genoeg is met meerdere gelijktijdige activiteiten
- Strikte timing-eisen heeft
- Op een gestructureerde manier moet kunnen groeien en onderhouden worden

Afbeelding: RTOS scheduling



Afbeelding: RTOS message queue



Overzicht per hoofdstuk – wat moet je echt kunnen?

• H1 – Microcontrollers & HAL

Begrijpen wat een microcontroller is (CPU, Flash, RAM, peripherals), waarom we HAL gebruiken

bovenop CMSIS, en hoe de **CubeMX** → **Keil** workflow eruitziet.

- **H2 – C voor embedded**

C-strings met `\0`, pointers en structs, `const` en `volatile`, en waarom HAL overal met **handles/pointers** werkt.

- **H3 – GPIO**

Input/output, pull-up/down, actief hoog/laag, en hoe je een knop of LED correct configureert in CubeMX (incl. debouncing-idee).

- **H4 – Polling/Interrupt/DMA**

Verschillen in CPU-belasting en latentie, en kunnen uitleggen wanneer je welke techniek kiest.

- **H5 – Timers & PWM**

Relatie tussen klok, prescaler, ARR, en hoe je een servo of motor via PWM aanstuurt.

- **H6 – I²C**

Open-drain + pull-ups, adressering, start/stop/ACK, en waarom je bij lezen eerst een registeradres schrijft.

- **H7 – SPI**

Lijnen MOSI/MISO/SCK/CS, CPOL/CPHA, verschillende topologieën (point-to-point, ster, daisy chain).

- **H8 – UART**

Frameopbouw (start, data, pariteit, stop), baudrate-afspraken en typische fouten.

- **H9 – WiFi (ESP32-C3)**

Rol van de ESP32 als modem met eigen TCP/IP-stack en basisidee van AT-commando's.

- **H10 – RTOS**

Taken, prioriteiten, scheduling, message queues/semaforen en waarom dit beter schaalbaar is dan één grote superloop.

Begrippenlijst (examengericht)

ADC (Analog-to-Digital Converter)

Zet een analoge spanning om naar een digitale waarde. Wordt vaak in combinatie met DMA gebruikt om continu samples in een buffer te plaatsen.

ARR (Auto-Reload Register)

Register in een timer dat bepaalt **tot welke waarde** de teller loopt. Samen met de prescaler bepaalt het de update-/PWM-frequentie.

Baudrate

Aantal **symbool-overgangen per seconde** bij seriële communicatie (UART). In onze context $\approx$ aantal bits per seconde. Voorbeeld: 115200 baud.

CMSIS (Cortex Microcontroller Software Interface Standard)

STandaardlaag met **C-structs naar de hardware-registers**. Laag niveau, snel en flexibel, maar minder leesbaar en minder portable dan HAL.

DMA (Direct Memory Access)

Hardware-module die **data kopieert tussen peripheral en geheugen** zonder dat de CPU elke byte zelf moet lezen/schrijven. CPU krijgt enkel interrupts bij half/vol buffer.

Duty cycle

Bij PWM: percentage van de periode dat het signaal **hoog** is. Bepaalt onder andere gemiddelde spanning en

servopositie.

GPIO (General-Purpose Input/Output)

Algemene in-/uitgangspinnen van de microcontroller. Kunnen geconfigureerd worden als input, output, alternate function (UART/SPI/I²C, ...) of analog.

HAL (Hardware Abstraction Layer)

Bibliotheek bovenop CMSIS die hardwaredetails verbergt en werkt met **handles en configuratiestructs**. Maakt code leesbaarder en makkelijk te hergebruiken.

ISR (Interrupt Service Routine)

Functie die automatisch wordt uitgevoerd wanneer een interrupt optreedt. Moet kort zijn en meestal enkel flags zetten of data bufferen.

I²C (Inter-Integrated Circuit)

Synchrone seriële bus met twee lijnen (SDA/SCL), open-drain met pull-ups, meerdere masters en slaves, **adresgebaseerd**.

Message queue

RTOS-mechanisme om **berichten/structs FIFO** tussen taken door te geven. Verhoogt loskoppeling en maakt timing voorspelbaar.

Mutex (Mutual Exclusion)

RTOS-mechanisme om ervoor te zorgen dat **slechts één taak tegelijk** een gedeelde resource (bv. een globale variabele of peripheral) gebruikt.

NVIC (Nested Vectored Interrupt Controller)

Onderdeel van de Cortex-M-core dat interruptbronnen beheert, prioriteiten toekent en nested interrupts mogelijk maakt.

Polling

Techniek waarbij de CPU in een lus **continu een statusbit controleert**. Simpel maar CPU-intensief.

Prescaler

Deler voor de timerklok. Verlaagt de ingangsklok zodat de teller trager loopt en bruikbare frequenties ontstaan.

PWM (Pulse Width Modulation)

Techniek waarbij de gemiddelde spanning wordt geregeld door de **pulsbreedte** (duty cycle) te variëren bij vaste frequentie.

RTOS (Real-Time Operating System)

Besturingssysteem voor microcontrollers dat **meerdere taken** plant op basis van prioriteiten en tijdseisen (real-time gedrag).

Semaphore

RTOS-mechanisme om **beschikbaarheid van een resource of event** aan te geven (bijvoorbeeld "er is een nieuw datapakket"). Lijkt op een teller die door taken wordt gepend/vrijgegeven.

SPI (Serial Peripheral Interface)

Snelle synchrone bus met lijnen MOSI/MISO/SCK en per slave een CS-lijn. Geen adressen op de bus;

full-duplex per klokpuls.

Superloop

Eenvoudige main-structuur met één grote `while(1)` waarin alle functionaliteit in volgorde wordt uitgevoerd. Wordt snel onoverzichtelijk bij complexe toepassingen.

Timer

Hardwareblok dat telt op basis van een klok. Kan interrupts genereren, PWM-signalen maken, of tijdsmetingen doen via Input Capture.

UART (Universal Asynchronous Receiver/Transmitter)

Asynchrone seriële communicatie met TX/RX en vooraf afgesproken baudrate, databits, pariteit en stopbits (bv. 115200 8N1).

Volatile

C-keyword dat aangeeft dat een variabele **asynchroon kan veranderen** (door hardware, interrupt, DMA). Verhindert gevaarlijke compiler-optimalisaties.

Actief hoog / actief laag

Geeft aan of een signaal **actief wordt bij 1 (hoog)** of bij **0 (laag)**. Actief laag wordt vaak met een streepje of `_N` aangeduid (bijv. `RESET_N`).

Callback

Functie die je naam geeft, maar die door de **HAL of het RTOS** wordt opgeroepen bij een event (bijv. `HAL_UART_RxCpltCallback`). Je implementeert de logica, het framework beslist **wanneer** ze wordt uitgevoerd.

Context switch

Omschakelen van de CPU van de ene taak/ISR naar de andere. Het RTOS moet hierbij **registers en stackpointers bewaren/herstellen**, wat tijd kost.

Full-duplex / half-duplex

Bij full-duplex kan **tegelijk** worden gezonden en ontvangen (klassieke SPI, UART), bij half-duplex niet (zenden/ontvangen om de beurt op dezelfde lijn).

Handle

Struct in HAL (bijv. `UART_HandleTypeDef`) die alle **status en configuratie** van een peripheral bijhoudt. Wordt altijd **by reference** (als pointer) doorgegeven aan HAL-functies.

Master / slave

Bij bussen als SPI en I²C: de **master** initieert communicatie en stuurt de klok; de **slave** reageert (heeft adres of CS-lijn).

Open-drain

Uitgangstype dat alleen actief naar **laag** kan trekken. Hoog wordt via een **pull-up** verkregen. Essentieel voor gedeelde bussen zoals I²C.

Pariteit

Extra bit in een UART-frame waarmee je een eenvoudige **foutdetectie** kan doen (even/oneven aantal "1"-bits).

Pull-up / pull-down

Weerstand die een ingang naar **VCC (pull-up)** of **GND (pull-down)** trekt wanneer er geen actieve stuurbron is, zodat de ingang niet "zweeft".

Ringbuffer (circular buffer)

Bufferstructuur waarbij begin en einde aan elkaar worden gelust. Veel gebruikt met **DMA** en UART om continu data te kunnen opslaan zonder dat de buffer hoeft te schuiven.

Taak / thread (RTOS-task)

Lopend programma binnen een RTOS met eigen stack en prioriteit. Meerdere taken lijken "parallel" te lopen doordat de scheduler ertussen wisselt.